Interactive Visualizations

with Plotly & Altair

Module: Data Analysis & Visualization | **Level:** Intermediate | **Duration:** 3 sessions · 4.5 hours

Module overview

This module teaches students how to build interactive, publication-quality data visualizations using two of Python's most powerful charting libraries. By the end of the module, students will be able to choose the right tool for a given task, add interactivity such as tooltips, dropdowns, and brush selections, and deploy charts inside notebooks and simple web apps.

Learning objectives

- Distinguish when to use Plotly versus Altair based on chart type and interactivity requirements
- Build charts with hover tooltips, dropdown filters, range sliders, and animated frames
- Apply the grammar-of-graphics model in Altair using marks, encodings, and transforms
- Chain and layer Altair charts to create linked, multi-view dashboards
- Export and embed charts as standalone HTML files or inside a Dash application
- Critique visualizations for clarity, colour accessibility, and honest data representation

Prerequisites

- Comfortable with pandas DataFrames (indexing, groupby, merge)
- Basic Matplotlib knowledge (figures, axes, plot types)
- Python functions, list comprehensions, and basic OOP
- Jupyter Notebook or JupyterLab environment

Required libraries

```
pip install plotly dash altair vega_datasets pandas
```

Plotly vs Altair: choosing the right tool

Both libraries produce interactive browser-based charts, but they follow different design philosophies. Plotly is imperative and explicit; Altair is declarative and concise. Understanding the trade-offs helps students pick the right tool for each job.

Plotly	Altair
Imperative, layer-by-layer API	Declarative grammar-of-graphics
Strong 3D & scientific chart support	Best for statistical & exploratory charts
Works natively with Dash for web apps	Built-in linked selections between charts
plotly.express for fast one-liners	Concise, composable chart specifications
Verbose but very explicit configuration	Compact syntax; complex charts = short code
Richer animation support (frames)	Renders via Vega-Lite in notebooks & HTML

Session 1 of 3 · 90 minutes

Plotly fundamentals

Plotly Express (px) is the high-level wrapper around Plotly's object-oriented core. It creates complete, interactive figures with a single function call. Students learn to build the three most common chart types, customise hover templates, and refine layout and colour mappings.

1.1 Scatter plot with hover tooltips

The scatter plot is the best starting point because it introduces the core Plotly Express pattern: pass a DataFrame, map columns to visual channels (x, y, color, size), and receive a fully interactive figure.

> Dataset: Gapminder — gdpPercap (x), lifeExp (y), pop (bubble size), continent (colour)
> Interaction: hover to see country name, GDP, and life expectancy

```
import plotly.express as px
import pandas as pd

# Load the built-in Gapminder dataset (one row per country per year)
df = px.data.gapminder()

# Filter to a single year so the chart is not cluttered
df_2007 = df[df['year'] == 2007]

# px.scatter creates an interactive scatter plot
# x, y       -> map DataFrame columns to axes
# color      -> colour each point by continent
# size       -> scale point area by population
# hover_name -> show country name in the tooltip header
# hover_data -> add extra columns to the tooltip
# log_x      -> use log scale on x axis (GDP spans orders of magnitude)
fig = px.scatter(
    df_2007,
    x='gdpPercap',
    y='lifeExp',
    color='continent',
    size='pop',
    hover_name='country',
    hover_data={'gdpPercap': ':,.0f', 'lifeExp': ':.1f', 'pop': ':,.0f'},
    log_x=True,
    title='GDP per capita vs Life expectancy (2007)',
    labels={'gdpPercap': 'GDP per capita (USD)', 'lifeExp': 'Life expectancy
(years)'}
)
```

```
# update_layout controls the overall figure appearance
fig.update_layout(
    height=520,
    legend_title_text='Continent',
    font=dict(family='Arial', size=13)
)

fig.show()  # opens in browser or renders in notebook
```

Key concepts demonstrated:

- Column-to-channel mapping: x, y, color, size, hover_name
- Log scale on a numeric axis via log_x=True
- Custom hover_data format strings (Python format mini-language)
- Layout customisation with update_layout()

1.2 Bar chart with sorted categories

Bar charts suit comparisons across a categorical dimension. Sorting bars by value is a small change with a large readability payoff.

```
import plotly.express as px

df = px.data.gapminder()

# Calculate mean life expectancy per continent in 2007
df_avg = (
    df[df['year'] == 2007]
    .groupby('continent', as_index=False)['lifeExp']
    .mean()
    .sort_values('lifeExp')  # sort so bars read low-to-high
)

# color_discrete_map assigns a fixed colour to each continent
# text_auto=True adds the bar value as a label on each bar
fig = px.bar(
    df_avg,
    x='lifeExp',
    y='continent',
    orientation='h',          # horizontal bars suit long category labels
    color='continent',
    text_auto='.1f',          # format bar labels to 1 decimal place
    title='Average life expectancy by continent (2007)',
    labels={'lifeExp': 'Avg life expectancy (years)', 'continent': ''}
)

# update_traces applies styling to every bar (marker = the bar shape)
```

```
fig.update_traces(textposition='outside')

fig.update_layout(
    showlegend=False,          # colour already encodes continent; legend is
redundant
    xaxis_range=[40, 85],      # fix axis range so small differences are
visible
    height=380
)

fig.show()
```

Key concepts demonstrated:

- Groupby aggregation feeding directly into a chart
- Horizontal orientation (orientation='h') for long labels
- text_auto for automatic data labels
- Removing a redundant legend (showlegend=False)

1.3 Line chart with multiple series

Line charts are the natural choice for time series. Plotly Express makes adding multiple lines from a tidy DataFrame trivial.

```
import plotly.express as px

df = px.data.gapminder()

# Filter to a subset of countries for a readable chart
countries = ['China', 'India', 'United States', 'Brazil', 'Nigeria']
df_sub = df[df['country'].isin(countries)]

# color='country' creates one line per country automatically
# markers=True adds a dot at each data point
fig = px.line(
    df_sub,
    x='year',
    y='lifeExp',
    color='country',
    markers=True,
    title='Life expectancy trends 1952-2007',
    labels={'year': 'Year', 'lifeExp': 'Life expectancy (years)', 'country':
'Country'}
)

# update_xaxes adds tick marks for every data year
fig.update_xaxes(tickvals=df['year'].unique())
```

```
# hovermode='x unified' shows all countries at once when hovering
fig.update_layout(
    hovermode='x unified',
    height=460,
    legend=dict(orientation='h', yanchor='bottom', y=1.02, xanchor='right',
x=1)
)

fig.show()
```

Key concepts demonstrated:

- Tidy data (one row per observation) as the natural Plotly input format
- hovermode='x unified' for comparing values across series
- Legend repositioning to avoid overlap with the chart area

Session 2 of 3 · 90 minutes

Plotly widgets & Dash

Real interactivity means the chart updates when the user makes a selection. This session introduces two approaches: ipywidgets (lightweight, notebook-only) for quick experimentation, and Dash (a full web-application framework) for shareable, deployable dashboards.

2.1 Dropdown filter with ipywidgets

ipywidgets are native Jupyter controls. Wrapping a plotting function with @interact creates a dropdown that re-renders the chart on every change — no JavaScript required.

> Prerequisite: pip install ipywidgets
> Run in a Jupyter Notebook; does not work in plain Python scripts

```
import plotly.express as px
import ipywidgets as widgets
from ipywidgets import interact

df = px.data.gapminder()

# interact() calls this function every time the widget value changes
# The 'continent' parameter becomes a dropdown populated from the list
@interact(
    continent=list(df['continent'].unique()),
    year=widgets.IntSlider(min=1952, max=2007, step=5, value=2007)
)
def plot_continent(continent, year):
    # Filter data to the selected continent and year
    filtered = df[(df['continent'] == continent) & (df['year'] == year)]

    fig = px.scatter(
        filtered,
        x='gdpPercap',
        y='lifeExp',
        size='pop',
        hover_name='country',
        log_x=True,
        title=f'{continent} — {year}',
        labels={'gdpPercap': 'GDP per capita', 'lifeExp': 'Life expectancy'}
    )
    fig.update_layout(height=420)
    fig.show()
```

Key concepts demonstrated:

- @interact decorator converts function parameters into widgets automatically
- IntSlider for numeric ranges; list for categorical dropdowns
- f-string chart titles that update to reflect the current selection

2.2 Animated chart with a play button

Plotly Express supports frame-based animation via the animation_frame parameter. This creates a play button and a slider so students can animate over time or any other ordered dimension.

```
import plotly.express as px

df = px.data.gapminder()

# animation_frame  -> which column defines each frame
# animation_group  -> keeps a consistent point identity across frames
#                     (without this, points jump around as years change)
# range_x / range_y -> fix axis ranges so the axes don't rescale each frame
fig = px.scatter(
    df,
    x='gdpPercap',
    y='lifeExp',
    animation_frame='year',
    animation_group='country',
    color='continent',
    size='pop',
    hover_name='country',
    log_x=True,
    range_x=[200, 100000],
    range_y=[25, 90],
    title='World development 1952–2007',
    labels={'gdpPercap': 'GDP per capita (log)', 'lifeExp': 'Life expectancy'}
)

# Slow the animation down so changes are visible (ms per frame)
fig.layout.updatemenus[0].buttons[0].args[1]['frame']['duration'] = 600
fig.layout.updatemenus[0].buttons[0].args[1]['transition']['duration'] = 300

fig.update_layout(height=540)
fig.show()
```

Key concepts demonstrated:

- animation_frame and animation_group for time-lapse charts
- Fixed axis ranges (range_x, range_y) for fair frame-to-frame comparison
- Accessing the play-button menu via fig.layout.updatemenus to adjust speed

2.3 Minimal Dash application

Dash turns a Python script into a web application. The dcc.Dropdown component sends its current value to a callback function, which returns an updated figure. This pattern scales to any number of inputs and outputs.

> Run as a Python script (not inside a Jupyter cell): python app.py
> Then open http://127.0.0.1:8050 in your browser

```
import dash
from dash import dcc, html, Input, Output
import plotly.express as px

df = px.data.gapminder()
continents = sorted(df['continent'].unique())

# Initialise the Dash application
app = dash.Dash(__name__)

# Layout describes the page structure using Python objects
# html.* maps to HTML tags; dcc.* (Dash Core Components) are interactive
widgets
app.layout = html.Div([

    html.H2('Gapminder explorer', style={'fontFamily': 'Arial'}),

    # Dropdown component: id links it to the callback below
    dcc.Dropdown(
        id='continent-dropdown',
        options=[{'label': c, 'value': c} for c in continents],
        value='Europe',          # default selection
        clearable=False,
        style={'width': '240px', 'marginBottom': '12px'}
    ),

    # Graph component: its 'figure' property is updated by the callback
    dcc.Graph(id='scatter-chart')
])


# @callback links Input component values to Output component properties
# Dash calls this function automatically whenever the dropdown changes
@app.callback(
    Output('scatter-chart', 'figure'),   # update the figure property...
    Input('continent-dropdown', 'value')  # ...when this dropdown value
changes
)
def update_chart(selected_continent):
    filtered = df[
        (df['continent'] == selected_continent) & (df['year'] == 2007)
```

```
    ]
    fig = px.scatter(
        filtered,
        x='gdpPercap', y='lifeExp',
        size='pop', hover_name='country',
        log_x=True,
        title=f'{selected_continent} — 2007',
        labels={'gdpPercap': 'GDP per capita', 'lifeExp': 'Life expectancy'}
    )
    fig.update_layout(height=480)
    return fig  # Dash replaces the chart with the returned figure


if __name__ == '__main__':
    app.run(debug=True)  # debug=True enables hot reload while editing
```

Key concepts demonstrated:

- Dash layout as a Python object tree (html.Div, dcc.Dropdown, dcc.Graph)
- The Input / Output callback pattern: inputs trigger the function, output receives the return value
- debug=True enables automatic page reload on file save during development

Session 3 of 3 · 90 minutes

Altair: grammar of graphics & selections

Altair uses the Vega-Lite grammar: every chart is built by declaring which data columns map to which visual channels (encodings). This makes it easy to think about charts as structured specifications rather than sequences of drawing commands.

Core grammar concepts

The three building blocks of any Altair chart are:

- Mark — the geometric object drawn for each row (point, bar, line, area, rect …)
- Encoding — the mapping from a data column to a visual channel (x, y, color, size, tooltip …)
- Transform — data operations applied before encoding (filter, aggregate, calculate …)

3.1 Basic Altair scatter plot

The minimal Altair chart is three lines: data, mark, and encoding. Everything else is optional.

```
import altair as alt
from vega_datasets import data as vdata

# Load the Gapminder dataset via vega_datasets
source = vdata.gapminder_health_income()

# alt.Chart(data)            -> binds the dataset to this chart
# .mark_point(filled=True)   -> draw a filled circle for each row
# .encode(...)               -> map columns to visual channels
chart = (
    alt.Chart(source)
    .mark_point(filled=True, size=80)
    .encode(
        x=alt.X('income:Q',                  # :Q = quantitative (continuous
number)
                scale=alt.Scale(type='log'),  # log scale on the x axis
                title='Income per person (log scale)'),
        y=alt.Y('health:Q',
                scale=alt.Scale(zero=False),  # don't force y axis to start at
0
                title='Life expectancy (years)'),
        color=alt.Color('region:N',           # :N = nominal (unordered
category)
                        title='Region'),
        size=alt.Size('population:Q',
                      scale=alt.Scale(range=[50, 1000]),  # min and max pixel
```

```
area
                        title='Population'),
        tooltip=[                               # columns shown on hover
            alt.Tooltip('country:N', title='Country'),
            alt.Tooltip('income:Q', title='Income', format=',.0f'),
            alt.Tooltip('health:Q', title='Life exp', format='.1f'),
        ]
    )
    .properties(width=580, height=380, title='Income vs health by country')
    .interactive()  # enables pan and zoom with scroll/drag
)

chart
```

Type annotation shorthand in Altair:

Suffix	Data type
:Q (Quantitative)	Continuous numbers (GDP, temperature, price)
:N (Nominal)	Unordered categories (country, colour, name)
:O (Ordinal)	Ordered categories (rating, size: S/M/L/XL)
:T (Temporal)	Date/time values (datetime64, date strings)

3.2 Bar chart with inline aggregation

Altair can aggregate data inside the encoding specification, so you do not need to pre-aggregate in pandas.

```
import altair as alt
from vega_datasets import data as vdata

source = vdata.gapminder_health_income()

# aggregate='mean' computes the group mean inside Altair
# shorthand string syntax: 'mean(health):Q' is equivalent to the long form
chart = (
    alt.Chart(source)
    .mark_bar()
    .encode(
        x=alt.X('mean(health):Q', title='Average life expectancy'),
        y=alt.Y('region:N',
                sort='-x',          # sort bars by descending x value
                title=''),
        color=alt.Color('region:N', legend=None),  # legend=None removes the
legend
        tooltip=[
            alt.Tooltip('region:N', title='Region'),
```

```
            alt.Tooltip('mean(health):Q', title='Avg life exp', format='.1f'),
        ]
    )
    .properties(width=520, height=300, title='Average life expectancy by
region')
)

chart
```

Key concepts demonstrated:

- Inline aggregation using 'mean(column):Q' shorthand
- sort='-x' sorts the nominal axis by the quantitative value (descending)
- Removing redundant legends when colour already encodes the same variable as the axis

3.3 Linked brush selection

This is Altair's flagship feature. A selection defined on one chart automatically filters another chart in the same specification. No callbacks or JavaScript required.

> This is the pattern that makes Altair uniquely powerful for exploratory analysis.
> Draw a rectangle on the scatter plot to filter the histogram in real time.

```
import altair as alt
from vega_datasets import data as vdata

source = vdata.gapminder_health_income()

# 1. Define a brush selection (rectangle drag on the scatter plot)
#    'encodings' limits the brush to the x and y axes only
brush = alt.selection_interval(encodings=['x', 'y'])

# 2. Scatter plot: the brush is attached here (add_params)
#    condition() changes a property depending on whether a point is selected
scatter = (
    alt.Chart(source)
    .mark_point(filled=True)
    .encode(
        x=alt.X('income:Q', scale=alt.Scale(type='log'), title='Income
(log)'),
        y=alt.Y('health:Q', scale=alt.Scale(zero=False), title='Life
expectancy'),
        color=alt.condition(
            brush,
            alt.Color('region:N', legend=None),  # selected: keep colour
            alt.value('lightgrey')               # unselected: turn grey
```

```
        ),
        size=alt.condition(brush, alt.value(80), alt.value(30)),
        tooltip=['country:N', 'income:Q', 'health:Q']
    )
    .add_params(brush)   # attach the brush to this chart
    .properties(width=380, height=300, title='Click and drag to select')
)

# 3. Histogram: uses transform_filter to show only the brushed points
#    When nothing is selected, all points pass the filter
histogram = (
    alt.Chart(source)
    .mark_bar()
    .encode(
        x=alt.X('income:Q', bin=alt.Bin(maxbins=20), title='Income'),
        y=alt.Y('count()', title='Countries'),
        color=alt.Color('region:N', legend=None)
    )
    .transform_filter(brush)   # filter to selected points only
    .properties(width=380, height=300, title='Distribution of selected
countries')
)

# 4. Place charts side by side with the | (hconcat) operator
scatter | histogram
```

Key concepts demonstrated:

- alt.selection_interval() creates a rectangular brush
- alt.condition() encodes different values for selected vs unselected marks
- transform_filter(brush) restricts which rows are included in a chart
- The | operator (hconcat) places charts side by side; & (vconcat) stacks vertically

3.4 Faceted chart (small multiples)

Faceting replicates the same chart for each category, making it easy to compare distributions without overplotting.

```
import altair as alt
from vega_datasets import data as vdata

source = vdata.gapminder_health_income()

# .facet() wraps any chart in a small-multiples grid
# columns=3 arranges the panels in rows of 3
chart = (
    alt.Chart(source)
    .mark_point(filled=True, opacity=0.7)
    .encode(
```

```
        x=alt.X('income:Q', scale=alt.Scale(type='log'), title='Income
(log)'),
        y=alt.Y('health:Q', scale=alt.Scale(zero=False), title='Life
expectancy'),
        size=alt.Size('population:Q', legend=None),
        tooltip=['country:N', 'income:Q', 'health:Q']
    )
    .properties(width=220, height=180)  # size of each individual panel
    .facet(
        facet=alt.Facet('region:N', title='Region'),
        columns=3
    )
    .resolve_scale(y='independent')  # let each panel set its own y range
)

chart
```

Key concepts demonstrated:

- .facet() wraps any chart specification into a grid of panels
- columns= controls the number of panels per row
- resolve_scale(y='independent') lets each panel use its own axis range

Exercises

Exercises

Exercise 1 — Gapminder animated scatter (Plotly) [Guided]

Build on the Session 1 scatter plot by adding animation across years and a colour scale.

Instructions:

1. Load the full Gapminder dataset (all years) using px.data.gapminder()
2. Create an animated scatter plot with animation_frame='year' and animation_group='country'
3. Map continent to colour, population to size, and country to hover_name
4. Use log scale on the x axis and fix the axis ranges so they do not rescale between frames
5. Slow the animation to 600 ms per frame
6. Export the figure as a standalone HTML file using fig.write_html('gapminder.html')

Exercise 2 — Sales dashboard in Dash [Guided]

Build a small Dash app that lets users filter a bar chart by product category.

```
# Starter dataset (run this to generate sample_sales.csv)
import pandas as pd, numpy as np

np.random.seed(42)
df = pd.DataFrame({
    'month': pd.date_range('2023-01-01', periods=120, freq='D'),
    'category': np.random.choice(['Electronics', 'Clothing', 'Food', 'Books'],
120),
    'sales': np.random.randint(50, 500, 120),
    'region': np.random.choice(['North', 'South', 'East', 'West'], 120)
})
df.to_csv('sample_sales.csv', index=False)
```

Instructions:

7. Load sample_sales.csv and create a Dash layout with a Dropdown for category and a Graph
8. Write a callback that filters the DataFrame by selected category and returns a bar chart of daily sales
9. Add a second Dropdown for region and update the callback to filter by both
10. Add a dcc.DatePickerRange component and filter by date range

Exercise 3 — Altair linked views [Guided]

Extend the Session 3 brush selection example.

Instructions:

11. Start with the scatter | histogram layout from Session 3
12. Add a third chart below (use & to vconcat): a bar chart showing the count of selected countries per region
13. Change the brush to selection_point (click to select individual points, shift+click for multiple)
14. Add a text mark layer on the scatter that shows the country name for selected points only

Capstone project — Your own interactive dashboard

Choose a public dataset and build a 3-chart interactive dashboard that combines at least one Plotly chart and one Altair chart.

> Suggested datasets: NYC taxi trips (nyc.gov), World Bank indicators (data.worldbank.org), Spotify Charts (kaggle.com), OpenAQ air quality (openaq.org), or any domain you care about.

Requirements:

- At least one Plotly chart with hover tooltips and one interactive control (dropdown or slider)
- At least one Altair chart with a linked selection between two views
- A 1-page design rationale explaining your chart choices and colour decisions
- A 3-minute demo to the class showing the interactivity

Quick reference

Quick reference

Plotly Express cheat sheet

Task	Code
Scatter plot	px.scatter(df, x='a', y='b', color='c')
Bar chart	px.bar(df, x='cat', y='val', color='cat')
Line chart	px.line(df, x='date', y='val', color='group')
Histogram	px.histogram(df, x='val', nbins=30)
Box plot	px.box(df, x='cat', y='val')
Animated chart	px.scatter(..., animation_frame='year')
Log axis	px.scatter(..., log_x=True)
Fix axis range	px.scatter(..., range_x=[0, 100])
Save as HTML	fig.write_html('chart.html')
Save as PNG	fig.write_image('chart.png') # needs kaleido

Altair cheat sheet

Task	Code
Basic chart	alt.Chart(df).mark_point().encode(x='a:Q', y='b:Q')
Bar chart	alt.Chart(df).mark_bar().encode(x='cat:N', y='mean(val):Q')
Line chart	alt.Chart(df).mark_line().encode(x='date:T', y='val:Q')
Brush selection	brush = alt.selection_interval()
Filter by brush	.transform_filter(brush)
Colour on selection	alt.condition(brush, 'region:N', alt.value('grey'))
Side by side	chart_a | chart_b
Stack vertically	chart_a & chart_b
Small multiples	.facet('category:N', columns=3)
Pan & zoom	.interactive()

Save as HTML	chart.save('chart.html')

Further reading

- Plotly documentation: https://plotly.com/python/
- Dash documentation: https://dash.plotly.com
- Altair documentation: https://altair-viz.github.io
- Vega-Lite grammar reference: https://vega.github.io/vega-lite/
- Fundamentals of Data Visualization — Claus Wilke (free online)